Question 2: Motion or Memory?

Code: https://github.com/TahaRazzaq/DL4STG (folder `PA2/Question2`)

All numbers are on the 1,000 test videos unless stated otherwise. Learned models are reported as **mean (seed 0, seed 1)**. Hardware: **NVIDIA Tesla T4** (PyTorch 2.11).

1. Setup

Table 1: implementation checks

Check	Expected	Yours
(a) max|M(M(x)) − x|	0	0 (torch.equal = True)
(b) max|R^4(x) − x|	0	0 (torch.equal = True)
(c) max|MRM(x) − R^{-1}(x)|	0	0 (torch.equal = True)
(d) bright pixel of M(p) (frame, row, column)	(7, 10, 43)	(7, 10, 43)
(e) bright pixel of R(p) (frame, row, column)	(7, 43, 10)	(7, 43, 10)
(f) 1× input / target frames	1–5 / 6–10	1, 2, 3, 4, 5 / 6, 7, 8, 9, 10
(g) 2× input / target frames	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19
(h) 3× input / target frames	1, 4, 7, 10, 13 / 16, 19	1, 4, 7, 10, 13 / 16, 19

Figure 1: Farneback flow, first 50 training videos (950 frame pairs); 103,050 pixels kept, median speed 3.32 px/frame

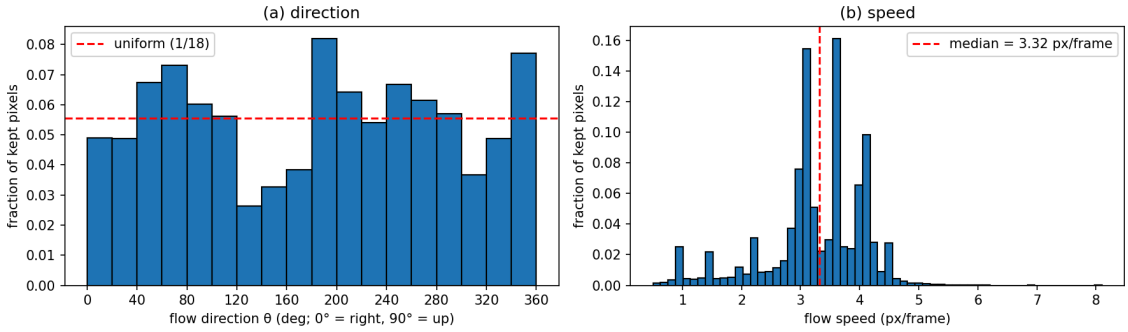


Figure 1: Farneback optical flow on the first 50 training videos (950 frame pairs). (a) Flow direction θ (0° = right, 90° = up), 18 bins of 20°, red line = uniform level 1/18. (b) Flow speed, red line = median. **Pixels kept: 103,050. Median speed: 3.32 px/frame.**

Q2.1

- **Direction (Fig. 1a):** every 20° bin holds between 0.026 and 0.082 of the pixels (uniform = 0.056). All directions occur in training.
- Mirroring maps θ → 180° − θ, and a 90° rotation maps θ → θ + 90°. Applying these to the histogram moves it only a little: the total-variation distance to its mirrored version is 0.132 and to its rotated version is 0.200. The bumps that remain come from having only ~100 digit trajectories in 50 videos.
- **So mirrored and rotated trajectories would already be familiar.** They only change how often each direction appears, and no direction is new.
- **Speed (Fig. 1b):** a typical speed is **3.32 px/frame** (10th–90th percentile 2.14–4.12). At 2× this becomes about **6.6 px/frame** and at 3× about **10.0 px/frame**. Almost no training pixel moves faster than 5 px/frame, so 2× and 3× motion is new to the model.
- **Overlap bias:** overlaps are a small share of the frame pairs, and wrong flow there mostly corrupts the speed (two motions get mixed). It does not favour any direction. It could add some low-speed values (the small peaks near 1 px/frame), but it cannot move the median from about 3.3 px/frame to 2× or 3× speeds. The answer does not change.

Q2.2

Transformation	Motion statistics	Digit appearance
Mirror M	Directions are relabelled (θ → 180° − θ), but the overall distribution stays almost the same (Q2.1). Speed is unchanged.	Changed (a mirrored 2 is not a 2). Symmetric digits such as 0, 1 and 8 change little.
Rotation R	Directions are relabelled (θ → θ + 90°), distribution stays almost the same. Speed is unchanged.	Changed more: every digit is turned on its side.
Speed-up k×	Changed: speed goes up k times, out of the training range.	Unchanged.

This makes a controlled experiment. Mirror and rotation change mainly the **appearance** while keeping the motion familiar, and speed-up changes only the **motion** while keeping the appearance familiar. So a failure under M/R points to a dependence on appearance, and a failure under speed-up points to the motion rule (unfamiliar speed or limited reach).

2. Baseline: ConvLSTM

Table 2: model A and the reference predictors (test set, 1×, t = 1)

Model	MSE_1	MSE_2	MSE_3	MSE_4	MSE_5	E(5)	Params
A	0.0172 (0.0175, 0.0170)	0.0205 (0.0208, 0.0203)	0.0228 (0.0231, 0.0225)	0.0245 (0.0248, 0.0242)	0.0262 (0.0264, 0.0259)	0.0222 (0.0225, 0.0220)	480,784

Model	MSE_1	MSE_2	MSE_3	MSE_4	MSE_5	E(5)	Params
Blank	0.0426	0.0426	0.0426	0.0425	0.0425	0.0426	0
Copy-last	0.0492	0.0604	0.0669	0.0708	0.0732	0.0641	0

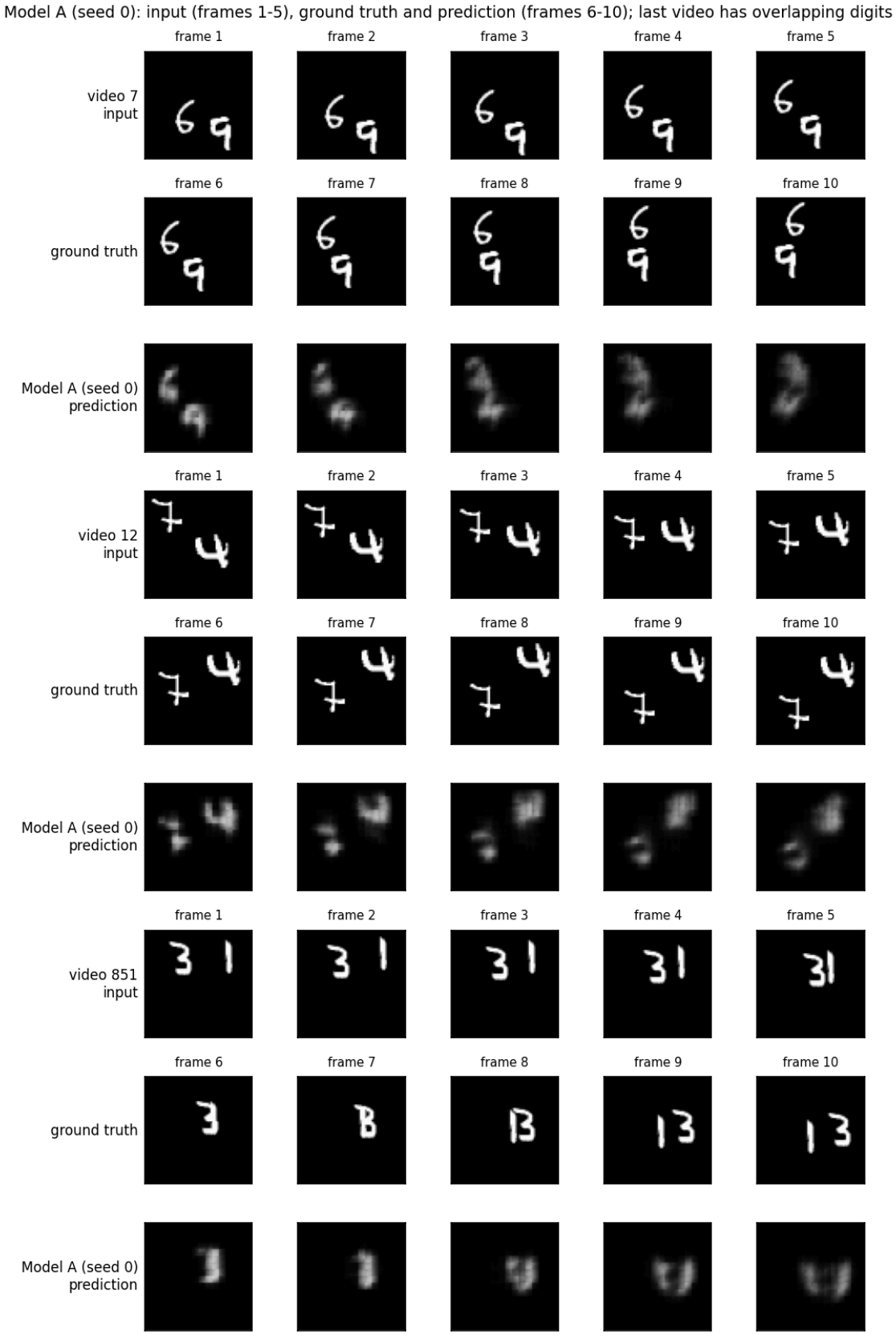


Figure 2: Model A (seed 0) on test videos 7, 12 and 851 (dataset indices 9007, 9012, 9851): input frames 1–5, ground truth 6–10 and prediction 6–10. In video 851 the digits overlap.

Q2.3 Reach of model A

1. `pixel_unshuffle(4)` : one cell of the 16 × 16 grid = 4 original pixels. The fold itself moves nothing.
2. Each 3 × 3 convolution (stride 1, padding 1) moves information by at most **1 cell** in each direction. The gate products (σ(f)⊙c, σ(o)⊙tanh(c)) are element-wise and move nothing. The 1 × 1 output convolution and `pixel_shuffle` move nothing.
3. Paths inside one step τ to $\hat{x}_{\tau+1}$:
   - u_τ → conv of layer 1 (1 cell) → $h^{(1)}_\tau$ → conv of layer 2 (1 cell) → $h^{(2)}_\tau$ → output: **2 cells**
   - $h^{(1)}_{\tau-1}$ → conv of layer 1 → conv of layer 2: **2 cells**
   - $c^{(1)}_{\tau-1}$ → $c^{(1)}_\tau$, $h^{(1)}_\tau$ (element-wise) → conv of layer 2: 1 cell
   - $h^{(2)}_{\tau-1}$ → conv of layer 2: 1 cell
   - $c^{(2)}_{\tau-1}$ → $c^{(2)}_\tau$ → $h^{(2)}_\tau$ (element-wise): 0 cells
4. Longest path = 2 cells × 4 px = **8 px per step** along a row or column ($8\sqrt{2} \approx 11.3$ px along a diagonal). The recurrence does not add to this: the state from step τ – 1 only enters through the same two convolutions.

A gradient check in the notebook gives the same per-source reach (8, 8, 4, 4, 0 px).

Q2.4

- Model A's E(5) = 0.0222 is about half of Blank (0.0426) and about a third of Copy-last (0.0641). It beats both at every horizon.
- Its error grows steadily, from MSE_1 = 0.0172 to MSE_5 = 0.0262 (+52 %), and each step adds less than the one before (+0.0033, +0.0023, +0.0017, +0.0017). Copy-last grows faster (0.0492 → 0.0732), and Blank stays flat.
- In Fig. 2 (videos 7, 12) the digits are in the right places at h = 1–2 but get blurrier with each horizon. By h = 5 they are soft blobs that still move the right way.
- **Overlap (video 851):** after the 3 and the 1 meet, the model does not separate them again. It predicts one merged blob near the meeting point. The 1 is lost and the blob turns into an unrecognisable "U" shape, while in the true frames the two digits come apart.

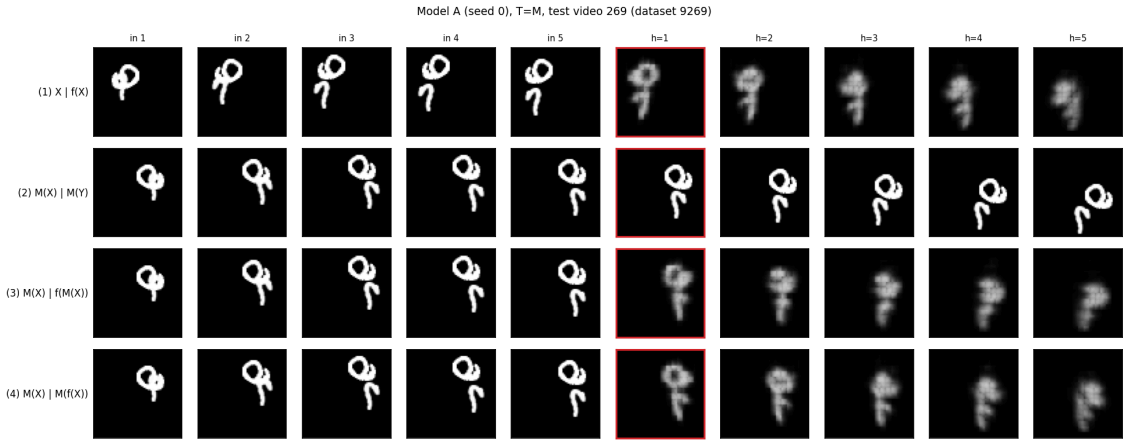
Q2.5 Predictions

- **Mirror test: pass.** Mirrored motion is already familiar (Q2.1), and many digits look almost the same mirrored, so C_M should be small and E_M close to E(5).
- **Rotation test: mostly pass, but worse than mirror.** The motion stays familiar, but rotated digits look new, so C_R and E_R should be noticeably higher than C_M and E_M.
- **Breaking speed: 3×.** At 2× digits move ≈ 6.6 px/frame, inside the 8 px reach, so the model should degrade but still track them. At 3× they move ≈ 10 px/frame, beyond the reach, so the predictions should fall apart (blur or stalled digits).

3. Testing the Baseline

Table 3: consistency gap and transformed error (1×, t = 1)

Model	E(5)	C_M	E_M	C_R	E_R
A	0.0222 (0.0225, 0.0220)	0.00163 (0.00155, 0.00171)	0.0228 (0.0231, 0.0226)	0.00288 (0.00254, 0.00321)	0.0240 (0.0240, 0.0239)
Blank	0.0426	0	0.0426	0	0.0426
Copy-last	0.0641	0	0.0641	0	0.0641



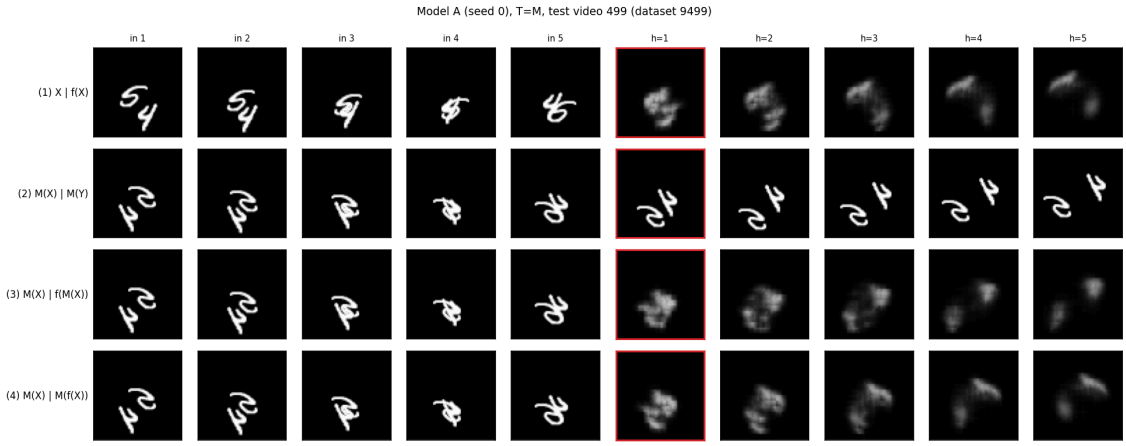




Figure 3: Model A (seed 0), the two test videos with the largest per-video C_M (a: video 269, C_M = 0.00432; b: video 499, C_M = 0.00379) and the two with the largest C_R (c: video 31, C_R = 0.00656; d: video 150, C_R = 0.00626). Rows: (1) X | f(X), (2) T(X) | T(Y), (3) T(X) | f(T(X)), (4) T(X) | T(f(X)). Columns 1–5 are inputs, columns 6–10 are horizons 1–5. GIFs: `PA2/Question2/pa2_q2/gifs/fig3_a_A_M_v269.gif`, `PA2/Question2/pa2_q2/gifs/fig3_b_A_M_v499.gif`, `PA2/Question2/pa2_q2/gifs/fig3_c_A_R_v31.gif`, `PA2/Question2/pa2_q2/gifs/fig3_d_A_R_v150.gif`.

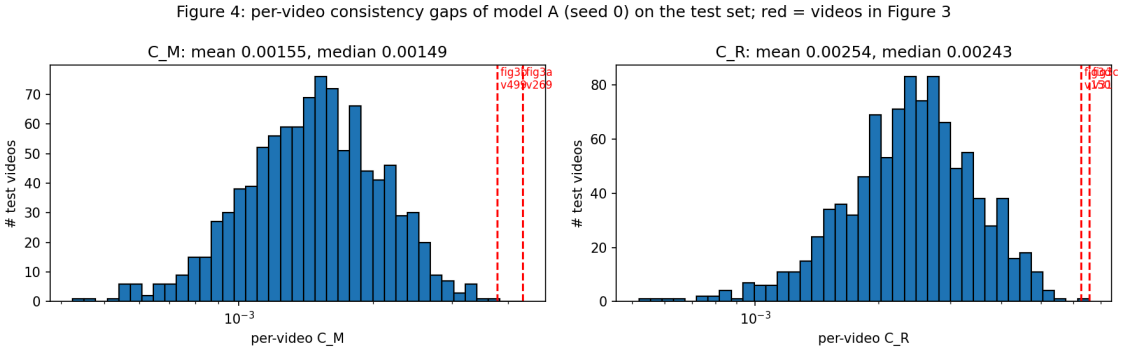


Figure 4: Histograms of per-video C_M (left) and C_R (right) of model A (seed 0) on the test set. Red lines mark the Figure 3 videos.

Q2.6

- A model that ignores its input, or copies it, is exactly consistent (C_T = 0) and still has a large error (Blank 0.0426, Copy-last 0.0641). So a low C_T only shows that the model treats original and transformed inputs the same way. It says nothing about whether the predictions are right.
- E_T measures correctness on the transformed clips. Only both together show a model that is consistent *and* accurate. For model A, C_M = 0.00163 is small, but E_M = 0.0228 shows that most of the error is ordinary forecasting error, not inconsistency.

Q2.7 Video 31, T = R (Fig. 3c)

- **Truth (row 2):** after rotation, the 5 moves up and to the right at the top, and the 3 slides left along the bottom.
- **Row 4, R(f(X)):** the lower digit stays at the bottom and smears sideways, and the upper digit becomes a horizontal streak at the top. The motion goes roughly the right way.
- **Row 3, f(R(X)):** the model sends the lower digit **the wrong way**. Instead of sliding left, part of it climbs up towards the centre (h = 3–5), while the rest fades at the bottom. By h = 4 the digit has split into two smears and **blurred into something unrecognisable**.
- So on the rotated input the model treats the lower digit as moving vertically, which is not how the same scene moves in the original orientation.

Q2.8

- Yes. For model A, C_R = 0.00288 is 1.8× C_M = 0.00163, and E_R = 0.0240 is above E_M = 0.0228 (E(5) = 0.0222).
- **Motion:** both transformations only relabel directions, and every direction is already common in training (Q2.1). The rotated histogram is a bit further from the original (TV distance 0.200 vs 0.132), but no rotated direction is rare (smallest bin 0.026). This is too small to explain the gap.
- **Appearance:** mirroring leaves many digits (0, 1, 8, and partly 3) almost unchanged, while rotation turns every digit on its side into shapes never seen in training (Q2.2). This is the bigger change. **The extra difficulty of rotation comes mainly from appearance.**

Table 4: model A at higher speeds (t = 1)

Model	E(2) 1×	E(2) 2×	E(2) 3×	E(5) 1×	E(5) 2×
A	0.0189 (0.0191, 0.0186)	0.0442 (0.0435, 0.0449)	0.0511 (0.0509, 0.0514)	0.0222 (0.0225, 0.0220)	0.0484 (0.0476, 0.0493)

Model	E(2) 1×	E(2) 2×	E(2) 3×	E(5) 1×	E(5) 2×
Copy-last	0.0548	0.0661	0.0703	0.0641	0.0719

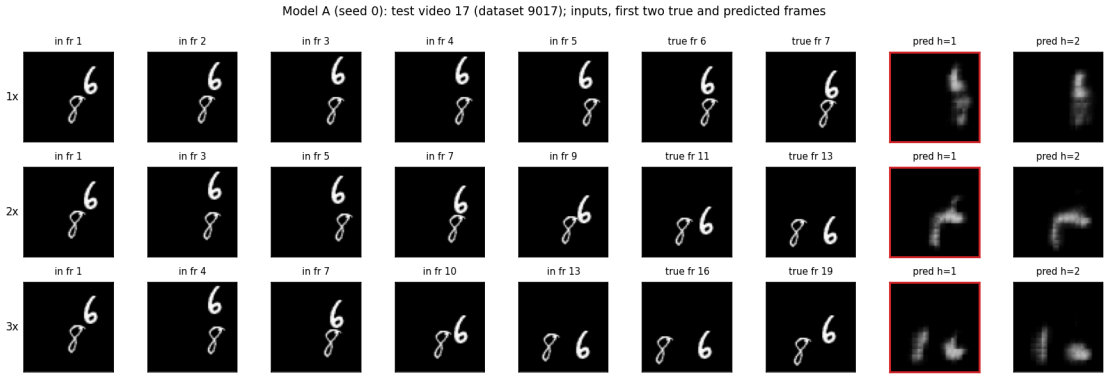


Figure 5: Model A (seed 0), test video 17 (dataset 9017) at 1×, 2× and 3×: five input frames, the first two true future frames and the first two predictions. The digits do not overlap in any displayed true frame.

Q2.9

- **Model A falls apart at 2×.** E(2) goes from 0.0189 at 1× to 0.0442 at 2× (2.3×) and 0.0511 at 3×. At 2× it is already worse than predicting a blank frame (≈ 0.043), though still better than Copy-last (0.0661).
- **This does not agree with the reach estimate.** From 8 px reach and Fig. 1, the expected breakdown was 3× (≈ 10 px/frame > 8 px), not 2× (≈ 6.6 px/frame < 8 px). What the estimate missed:
  - Reach is the *largest possible* distance. Most of a convolution's weight sits near the centre of the kernel, and the gates multiply signals, so the distance the model actually uses is smaller than 8 px.
  - Reach only says the information *can* get there. The model also has to have *learned* to move digits that far, and it never saw speeds above ≈ 5 px/frame (unfamiliar motion).
- **What the failure looks like (Fig. 5):**
  - **2×:** the 6 and the 8 are not moved apart. They **blur together into one streak** along their recent path, close to where they were in the last input frame. The model is not extrapolating the larger step. It hedges, spreading intensity over the places the digits could be, which is what an MSE-trained model does when it is unsure.
  - **3×:** the two digits **stall** near their last input positions and lose their shape (the 6 becomes a **blob**). The model seems to apply roughly its familiar 1× step and blur the result, rather than follow the true jump of ≈ 10 px.

Q2.10

- No. Both explanations predict the same thing: model A only ever saw 1× motion, *and* 2×/3× steps approach or exceed its reach. From Table 4 alone they cannot be told apart.
- **Experiment:** keep the architecture fixed and train with speed augmentation (k ∈ {1, 2, 3}) (model C).
  - If the speed failure goes away, it was **unfamiliar motion**.
  - If it remains at speeds whose per-step displacement exceeds the reach, train a model with a larger reach on the same augmented data (model D). If D fixes what C cannot, the failure is **architectural reach**.

Q2.11 Predictions

Model B will close most of the mirror and rotation gap, because that failure comes from digit appearance. Model C will fix the 2× failure, because 2× motion is within reach and only unfamiliar, but it will still fail at 3×. Only model D, with its larger reach, will fix 3×.

4. What Can Data Fix?

Table 5: models A–D (and the optional D+B), test set

Model	E(5) 1×	C_M	E_M	C_R	E_R	E(2) 1×	E(2) 2×	E(2) 3×	
A	0.0222 (0.0225, 0.0220)	0.00163 (0.00155, 0.00171)	0.0228 (0.0231, 0.0226)	0.00288 (0.00254, 0.00321)	0.0240 (0.0240, 0.0239)	0.0189 (0.0191, 0.0186)	0.0442 (0.0435, 0.0449)	0.0511 (0.0509, 0.0514)	4
B	0.0229 (0.0233, 0.0224)	0.00111 (0.00106, 0.00115)	0.0229 (0.0234, 0.0224)	0.00134 (0.00141, 0.00128)	0.0228 (0.0232, 0.0223)	0.0192 (0.0197, 0.0187)	0.0431 (0.0433, 0.0429)	0.0490 (0.0496, 0.0483)	4
C	0.0267 (0.0272, 0.0263)	0.00135 (0.00117, 0.00154)	0.0269 (0.0273, 0.0265)	0.00258 (0.00220, 0.00295)	0.0279 (0.0282, 0.0276)	0.0232 (0.0237, 0.0227)	0.0288 (0.0292, 0.0285)	0.0333 (0.0335, 0.0331)	4
D	0.0299 (0.0308, 0.0290)	5.00e-04 (4.73e-04, 5.26e-04)	0.0300 (0.0308, 0.0292)	0.00118 (9.19e-04, 0.00144)	0.0306 (0.0312, 0.0300)	0.0266 (0.0277, 0.0255)	0.0298 (0.0309, 0.0287)	0.0328 (0.0336, 0.0319)	6
D+B	0.0306 (0.0331, 0.0282)	4.77e-04 (5.56e-05, 8.98e-04)	0.0306 (0.0331, 0.0282)	4.43e-04 (6.33e-05, 8.22e-04)	0.0306 (0.0331, 0.0281)	0.0280 (0.0306, 0.0254)	0.0307 (0.0327, 0.0288)	0.0338 (0.0354, 0.0322)	6

Training time = mean wall-clock time per epoch over 30 epochs. Inference time = all 1,000 test videos (1×, t = 1), batch 32, Tesla T4.

Q2.12 Model D

- **Design:** identical to model A (4 × 4 patch folding, two ConvLSTM layers without peepholes, 1 × 1 output convolution, `pixel_shuffle` + sigmoid, 9-step rollout without teacher forcing) except that both ConvLSTM layers use **5 × 5 kernels** (stride 1, padding 2) and **44 hidden channels**.

- **Parameters:** layer 1: (16 + 44)·176·25 + 176 = 264,176; layer 2: (44 + 44)·176·25 + 176 = 387,376; output: 44·16 + 16 = 720. Total **652,272 ≤ 721,176**.

- **Selection (validation split only):** three candidates, all with 16 px reach, were each trained for 30 epochs (seed 0, speed augmentation, same protocol). The one with the lowest mean validation E(2) over 1×, 2× and 3× was chosen.

Candidate	Params	Val E(2) 1×	Val E(2) 2×	Val E(2) 3×	Mean	Leaves blank plateau at epoch
5 × 5 kernels, 44 channels (chosen)	652,272	0.0274	0.0306	0.0330	**0.0304**	19
3 × 3 kernels, dilation 2, 64 channels	480,784	0.0284	0.0308	0.0325	0.0306	19
8 × 8 patches, 3 × 3 kernels, 64 channels	594,496	0.0395	0.0395	0.0395	0.0395	never

  "Leaves blank plateau" = first epoch whose validation E(5) at 1× is below 0.9 × Blank (0.0418). The 8 × 8-patch model still predicts near-blank frames after 30 epochs.

- **Reach** (same conventions as Q2.3): each 5 × 5 convolution moves information by up to **2 cells**.

  - u_τ → conv of layer 1 (2 cells) → conv of layer 2 (2 cells) → output: **4 cells**
  - $h^{(1)}_{\tau-1}$ → conv of layer 1 → conv of layer 2: **4 cells**
  - $c^{(1)}$*{τ−1} → conv of layer 2: 2 cells; $h^{(2)}$*{τ−1} → conv of layer 2: 2 cells; $c^{(2)}_{\tau-1}$: 0
  - Longest path = 4 cells × 4 px = **16 px per step**, strictly larger than model A's 8 px (diagonal 22.6 px vs 11.3 px). This covers the ≈ 10 px/frame of 3× motion.

Q2.13

Failure of model A	Label	Supporting comparison (Table 5)
Inconsistent under rotation (and, less, under mirror)	**Appearance**	B (only symmetry augmentation, same architecture and speeds) halves C_R (0.00288 → 0.00134) and lowers C_M (0.00163 → 0.00111). E_R falls from 0.0240 to 0.0228, equal to B's own E(5) (0.0229). Since the motion directions were already familiar (Q2.1), what B adds is mainly new digit appearances.
Breaks down at 2×	**Unfamiliar motion**	C (only speed augmentation, same reach) cuts E(2) at 2× from 0.0442 to 0.0288. D, with twice the reach, gives 0.0298, no better.
Breaks down at 3×	**Unfamiliar motion** (no	C cuts E(2) at 3× from 0.0511 to 0.0333. D gives 0.0328, inside C's seed range (0.0331–0.0335 vs 0.0319–0.0336).

Failure of model A	Label	Supporting comparison (Table 5)
	evidence of reach)	

- In this run model C does **not** do badly at 3× compared with D, so the case in the question (C bad, D good) did not occur. If it had, it would mean that C had seen fast motion in training and still could not follow it, so the limit is the architecture's reach, not the data.
- One reason reach did not bind: 3× moves a digit ≈ 10 px per frame, which along a row or column is 7–10 px depending on direction, close to A's 8 px reach. Only part of the 3× motions are actually beyond it.
- **Why more data cannot carry information further than the architecture allows:** within one step, the prediction at a pixel is a function only of inputs and states within the reach. Anything further away is not in that computation, so no weights, whatever data they are trained on, can use it. A model trained on such data can only guess, which gives the blur seen in Fig. 5.
- Caveat: D starts learning much later than C. On validation it leaves the blank-prediction plateau at epoch 19 (seed 0) and 13 (seed 1), while C does so at epochs 4 and 6. So D gets only 11–17 useful epochs out of 30, and its seeds differ more (E(5) 0.0308 vs 0.0290). With equal useful training, D might still show a small reach benefit at 3×.

Q2.14

- Yes. E(5) at 1× rises from 0.0222 (A) to 0.0229 (B, +3 %), 0.0267 (C, +20 %) and 0.0299 (D, +35 %).
- **B:** a small cost for a halved C_R and lower E_R. Worth it.
- **C:** E(2) at 1× rises from 0.0189 to 0.0232, but E(2) at 2× and 3× falls by 35 % each (0.0442 → 0.0288, 0.0511 → 0.0333). The same capacity now has to cover three speeds. Worth it if the model has to handle varying speeds.
- **D:** worse than C at 1× with no gain at 2× or 3×. Not worth it in this run (partly because it starts learning late, Q2.13).

5. The Final Model

Why f̄ has C_M = C_R = 0. Take any $T \in G$. Each $g \in G$ acts linearly on frames (it only moves pixels), and $g \mapsto gT$ is a one-to-one map of G onto itself. Writing $g' = gT$, so $g^{-1} = T\, g'^{-1}$:

$$\bar{f}(T(X)) = \tfrac{1}{8} \Sigma_{g\in G}\, g^{-1}(f(gT(X))) = \tfrac{1}{8} \Sigma_{g'\in G}\, T\, g'^{-1}(f(g'(X))) = T(\tfrac{1}{8} \Sigma_{g'\in G}\, g'^{-1}(f(g'(X)))) = T(\bar{f}(X)).$$

So f(T(X)) = T(f(X)) exactly for any f, and C_T = MSE(f(T(X)), T(f(X))) = 0 for T = M and T = R. Also, since T only moves pixels, E_T = MSE(T(f(X)), T(Y)) = MSE(f(X), Y) = E(5).

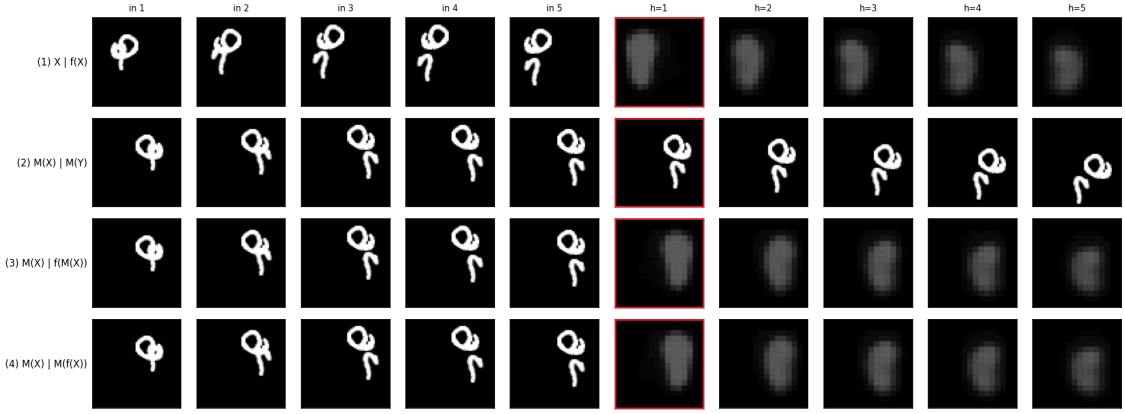
Final model choice: symmetrised D. Table 5 does not support D+B: it is no better than D on any error column (E(5) 0.0306 vs 0.0299, E(2) 3× 0.0338 vs 0.0328), and symmetrisation already makes D exactly consistent.

Table 6: symmetrised models (test set)

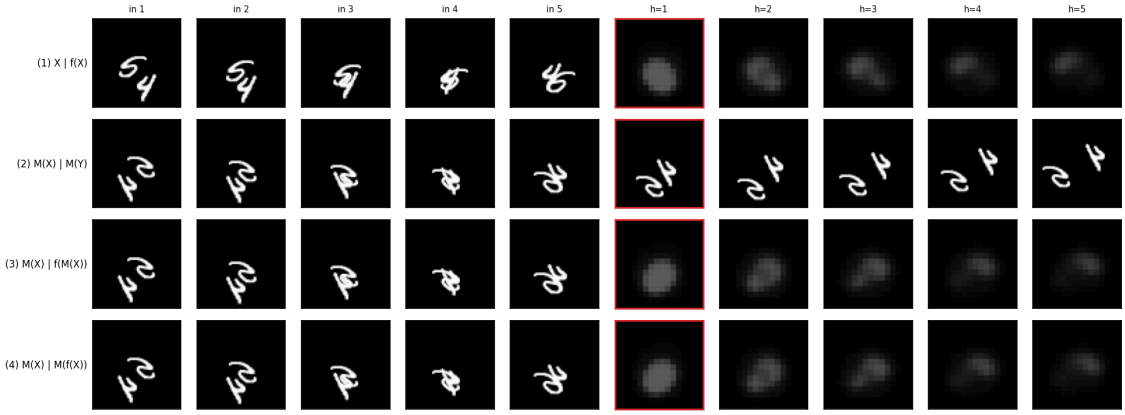
Model	E(5) 1×	C_M	E_M	C_R	E_R	E(2) 1×	E(2) 2×	E(2) 3×	Para
Sym(B)	0.0223 (0.0227, 0.0218)	7.63e-17 (8.24e-17, 7.03e-17)	0.0223 (0.0227, 0.0218)	6.19e-17 (6.69e-17, 5.68e-17)	0.0223 (0.0227, 0.0218)	0.0187 (0.0192, 0.0183)	0.0425 (0.0427, 0.0423)	0.0482 (0.0488, 0.0476)	480,7
Sym(D) (final)	0.0299 (0.0307, 0.0291)	3.64e-17 (3.31e-17, 3.97e-17)	0.0299 (0.0307, 0.0291)	2.84e-17 (2.70e-17, 2.98e-17)	0.0299 (0.0307, 0.0291)	0.0267 (0.0278, 0.0256)	0.0298 (0.0308, 0.0287)	0.0327 (0.0335, 0.0319)	652,2

Inference time includes all eight transformations, forward passes, inverse transformations and the averaging.

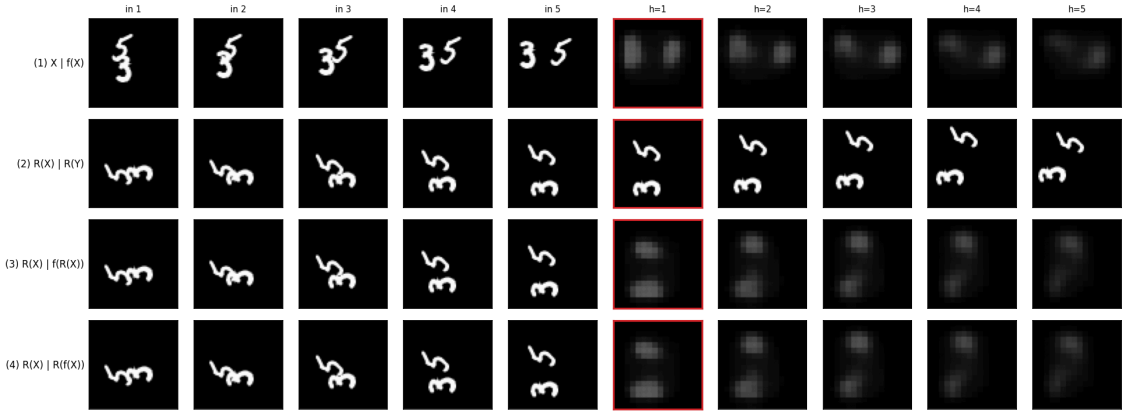
Sym(D) (seed 0), T=M, test video 269 (dataset 9269)



Sym(D) (seed 0), T=M, test video 499 (dataset 9499)



Sym(D) (seed 0), T=R, test video 31 (dataset 9031)



Sym(D) (seed 0), T=R, test video 150 (dataset 9150)

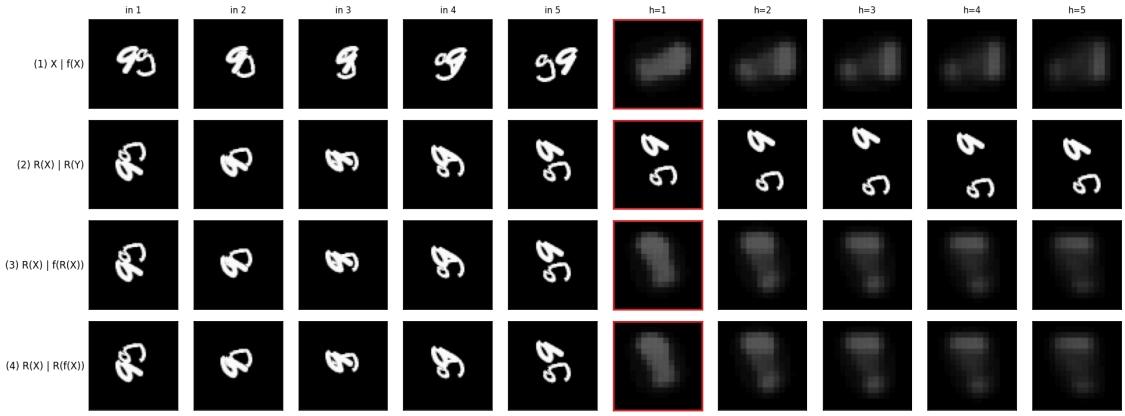


Figure 6(a): Figure 3 repeated with the final model, Sym(D) (seed 0), on the same videos: 269 (M), 499 (M), 31 (R), 150 (R). Same four rows as Figure 3. GIFs: `PA2/Question2/pa2_q2/gifs/fig6a_a_final_M_v269.gif`, `PA2/Question2/pa2_q2/gifs/fig6a_b_final_M_v499.gif`, `PA2/Question2/pa2_q2/gifs/fig6a_c_final_R_v31.gif`, `PA2/Question2/pa2_q2/gifs/fig6a_d_final_R_v150.gif`.

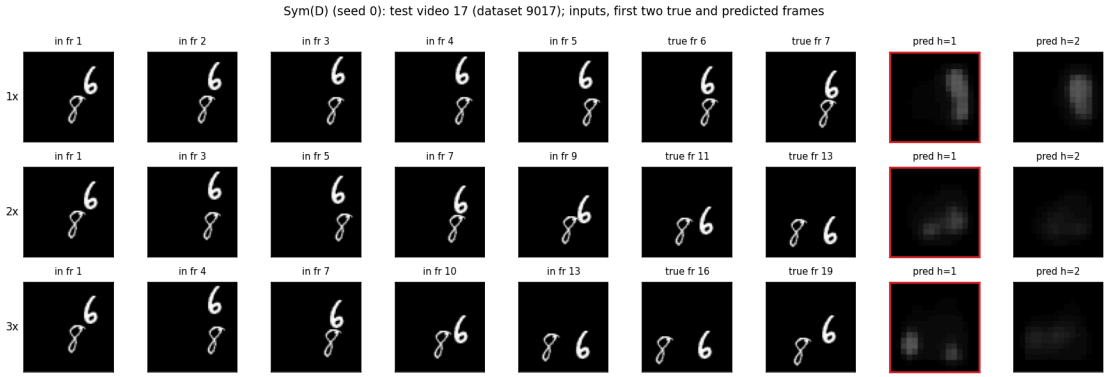


Figure 6(b): Figure 5 repeated with Sym(D) (seed 0) on the same test video 17 at 1×, 2× and 3×.

Q2.15

	Learned symmetry (B)	Built-in symmetry (Sym(B), Sym(D))
Consistency	Reduced but not zero: C_M = 0.00111, C_R = 0.00134	Zero up to floating-point error: C_M, C_R ≈ 3e-17 to 8e-17
Accuracy, original clips	E(5) = 0.0229	Sym(B) 0.0223 (better than B); Sym(D) 0.0299 (same as D)
Accuracy, transformed clips	E_M = 0.0229, E_R = 0.0228	E_M = E_R = E(5) exactly (Sym(B) 0.0223)
Inference cost	B 0.46 s, D 0.49 s	3.71 s for both (≈ 8×, one forward pass per symmetry)

- **Built-in symmetry is exact.** It holds by construction for any weights (proof above). Augmentation only pushes the weights towards consistency through the loss, so how close it gets depends on training.
- **When averaging can hurt:** the eight predictions agree only if f is already nearly equivariant. If f is much more accurate in the original orientation than in the others, or puts sharp digits in slightly different places in different orientations, the average is a blurred mix and can be worse than f alone on the original clips. (Because MSE is convex, the average can never be worse than the *mean* of the eight transformed errors.)
- **Here it did not hurt accuracy:** Sym(B) is better than B at every speed (E(5) 0.0223 vs 0.0229, E(2) 3× 0.0482 vs 0.0490), and Sym(D) equals D (0.0299). The

predictions do look softer (Fig. 6), but that does not show up as higher MSE.

Q2.16 Video 31, T = R (Fig. 6a, third panel)

- **Consistency: fixed.** Rows 3 and 4 are now identical (C_R = 2.73e-17 vs 0.00656 for model A). The lower digit no longer climbs upwards in only the rotated version. **Symmetrisation** fixed this part.
- **Correctness: not fixed.** E_R = 0.0336, the same as model A (0.0334). Both digits are predicted as soft, unrecognisable blobs that barely move: the lower digit does not clearly slide left and the upper one does not move up. This video is at 1×, where the speed augmentation and larger reach of D do not help and D is less accurate than A (E(5) 0.0299 vs 0.0222).

Q2.17 Final verdict

Only partly. The final model is exactly consistent under all eight symmetries, but that comes from the wrapper, not from learning. What it learned from data is better: speed augmentation made it follow 2× and 3× motion far better than model A (E(2) at 3×: 0.0327 vs 0.0511), and model B showed that seeing transformed digits reduces the dependence on appearance. Against that, its accuracy at 1× fell (E(5) 0.0299 vs 0.0222), and its predictions turn into blobs within two or three horizons. To convince a skeptic, the model would need to keep digits sharp and correctly placed over all five horizons, bounce them off the walls at the right frame, and keep this up at a speed it never trained on (e.g. 1.5× or 4×). One test it still has not passed: keeping two digits recognisable and separate through all five horizons (Fig. 6), especially after they overlap.